do280-4.18: Days remaining **89** | Lab hours used **4/80**

DO280-4.22: Days remaining **89** | Lab hours used **0/80**

Red Hat OpenShift Administration II: Configuring a Production Cluster

Course | Lab Environment

Lab: Secure Applications

Configure a project that requires custom settings.

Secure applications by encrypting and restricting network traffic.

Automate cluster maintenance tasks.

Outcomes

- Create a project quota.
- Create a limit range.
- Use role-based access control to grant permissions to service accounts and groups.
- Encrypt the traffic end-to-end with TLS by using a signed certificate.
- Restrict cluster internal traffic to pods by using network policies.
- Configure a cluster maintenance application to run periodically.



> **NOTE**
>
> The allocated time for this activity is 45 minutes. If you need additional time to complete the task, then you must revisit the course content, or practice more.

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start compreview-apps
```

The `lab` command copies the exercise files to the `~/DO0019L/labs/compreview-apps` directory and creates the `workshop-support` group with the `do280-support` user. The `do280-support` user has `redhat` as the password. The `lab` command also restores the project template configuration from the previous exercise.

Specifications

The company wants to host workshops on an OpenShift cluster while ensuring efficient resource management and automated cleanup of completed workshops. The company also wants to secure the application.

As the OpenShift cluster administrator, your responsibility is to set up a dedicated `workshop-support` namespace for the workshop support team. Create a namespace instead of a project to avoid using the project template. The project template applies a default configuration for workshop projects, and does not apply the configuration to the `workshop-support` namespace.

This dedicated namespace enables the support team to manage infrastructure services effectively and to deploy the applications without interfering with workshop projects.

Then, as a support team member, you configure and deploy a project cleaner application to delete completed workshop projects automatically.

Additionally, you must set up the social media API that attendees from all workshops use securely. The definition files for the API are in the `~/DO0019L/labs/compreview-apps/beeper-api` directory.

> **NOTE**
>
> The cluster API URL is `https://api.lab.example.com:6443`. The password for the `developer` user is `developer`, and the password for the `admin` user is `redhatocp`. If you use the OpenShift web console, then select **Red Hat Identity Management** on the login page.

- The `workshop-support` namespace must match the following criteria:

  - The `workshop-support` namespace has the `category=support` label.
  - The `workshop-support` group has the `admin` role in the cluster.
- Workloads from the `workshop-support` namespace must enforce the following constraints:
  - The project uses up to 4 CPUs.
  - The project uses up to 4 GiB of RAM.
  - The project requests up to 3.5 CPUs.
  - The project requests up to 3 GiB of RAM.
- The `workshop-support` namespace defines the default resource specification for workloads:
  - A default limit of 300m CPUs
  - A default limit of 400 MiB of RAM
  - A default request of 100m CPUs
  - A default request of 250 MiB of RAM

> **NOTE**
>
> Any quota or limit range must have the `workshop-support` name for grading purposes.

- The project cleaner application is defined in the `~/DO0019L/labs/compreview-apps/project-cleaner/cron-job.yaml` file. Edit the definition file so that the application matches the following criteria:
  - The project cleaner application uses a service account named `project-cleaner-sa`.
  - Create the `project-cleaner` cluster role to allow the `get`, `list`, and `delete` operations for the `namespace` resources.
  - Create a cluster role binding to grant the `project-cleaner` cluster role to the `project-cleaner-sa` service account in the `workshop-support` namespace.
  - Create a job that runs every minute to delete projects with the `workshop` label that exist for more than 10 seconds. This short expiration time is deliberate for this lab.
- Deploy the `beeper-db` database by using the `~/DO0019L/labs/compreview-apps/beeper-api/beeper-db.yaml` resource manifest in the `workshop-support` namespace.
- The social media API requires you to configure the application to match the following criteria:
  - The `beeper-api` application uses the `beeper-api.pem` certificate and the `beeper-api.key` private key in the `~/DO0019L/labs/compreview-apps/beeper-api/certs/` directory.

  - Create a TLS secret named `beeper-api-tls` to save the `beeper-api.pem` SSL certificate and the `beeper-api.key` private key.
  - The SSL certificate is signed with a custom CA certificate in the `~/DO0019L/labs/compreview-apps/beeper-api/certs/` directory.
- Edit the `~/DO0019L/labs/compreview-apps/beeper-api/deployment.yaml` file to configure the `beeper-api` deployment with the following settings:
  - Mount the `beeper-api-tls` secret in the `/etc/pki/beeper-api/` directory.
  - Set the `CERT_PATH` environment variable to `/etc/pki/beeper-api/tls.crt`.
  - Set the `CERT_KEY_PATH` environment variable to `/etc/pki/beeper-api/tls.key`.
  - Set the `TLS_ENABLED` environment variable to `true`.
  - Update the startup, readiness, and liveness probes to use HTTPS.
  - The `beeper-api` pods accept connections on port 8080.
- Create a service named `beeper-api` that exposes the `beeper-api` deployment on port 443.
- Expose the `beeper-api` service with a passthrough route named `beeper-api-route` that uses the `beeper-api.apps.lab.example.com` hostname.
- The API uses the following parameters to retrieve messages from the database:
  - API Endpoint: `https://beeper-api.apps.lab.example.com/api/beeps`
  - Output format: JSON
- The API uses the following JSON specification to save data:
  - API endpoint: `https://beeper-api.apps.lab.example.com/api/beep`
  - HTTP POST request
  - Content type: `application/json`
  - JSON payload: `{"username":"`*`my-user`*`","content":"`*`my content`*`"}`
- You can enter the database pod in the `workshop-support` namespace by using the `oc exec` command.
- You can list the databases by using the `psql -e --list` command inside the database container.
- You can get the contents of the `beep` table in the `beeper` database by using the `psql -d beeper -c "SELECT * FROM beep;"` command inside the database container.
- The security restrictions for the beeper database and API must match the following criteria:
  - The network policy for the `beeper-db` pods accepts ingress traffic to port 5432 from pods with the `app=beeper-api` label and in a namespace with the `category=support` label. The `workshop-support` namespace has the `category=support` label.

- The network policy for the `beeper-api` pods allows ingress traffic to port 8080 from the ingress router pods on namespaces with the `policy-group.network.openshift.io/ingress=""` label. The network policy template files are in the `~/DO0019L/labs/compreview-apps/beeper-api` directory.

> **NOTE**
>
> The `lab` command copies the solution files for the exercise to the `~/DO0019L/solutions/compreview-apps` directory.

1. Prepare the `workshop-support` namespace and grant privileges.
   1.1. Create the `workshop-support` namespace and add the `category=support` label.
   1.2. Use the `oc adm policy add-cluster-role-to-group` command to grant the `admin` role to the `workshop-support` group.
   1.3. Use the `oc create quota` command to create a quota with the following specification as a `hard` limit:

	CPU	Memory
Requests	`3500m`	`3Gi`
Limits	`4`	`4Gi`

   1.4. Create the limit range in the `workshop-support` namespace with the following specification:

	CPU	Memory
Default	`300m`	`400Mi`
Default request	`100m`	`250Mi`

2. Deploy the `project-cleaner` application.
   2.1. Use the `oc create serviceaccount` command to create the `project-cleaner-sa` service account in the `workshop-support` namespace.
   2.2. Create the `project-cleaner` cluster role to allow the `get`, `list`, and `delete` operations for the `namespace` resources.
   2.3. Use the `oc adm policy add-cluster-role-to-user` command to grant the `project-cleaner` cluster role to the `project-cleaner-sa` service account in the `workshop-support` namespace.

2.4. Edit the `~/DO0019L/labs/compreview-apps/project-cleaner/cron-job.yaml` file. Set the `schedule` field to `"*/1 * * * *"` to execute the cron job every minute. Set the `serviceAccountName` field to `project-cleaner-sa`.

3. Deploy the `beeper-api` application.

3.1. Deploy the database for the application by using the `~/DO0019L/labs/compreview-apps/beeper-api/beeper-db.yaml` resource manifest, which contains all the resources for the database application.

3.2. The `lab` command generates a SSL certificate that is signed with a custom CA. Create a TLS secret to store the SSL certificate and key, in the `~/DO0019L/labs/compreview-apps/beeper-api/certs` directory, by using the `oc create secret tls` command.

3.3. Use the following settings in the `beeper-api` deployment to configure TLS:

- Mount the `beeper-api-tls` secret in the `/etc/pki/beeper-api/` directory.
- Set the `CERT_PATH` environment variable to `/etc/pki/beeper-api/tls.crt`.
- Set the `CERT_KEY_PATH` environment variable to `/etc/pki/beeper-api/tls.key`.
- Set the `TLS_ENABLED` environment variable to `true`.
- Update the startup, readiness, and liveness probes to use HTTPS.

Edit the `~/DO0019L/labs/compreview-apps/beeper-api/deployment.yaml` resource manifest according to the specification and then create the deployment in the `workshop-support` namespace.

3.4. Use the `oc expose` command to create a `ClusterIP` service that exposes the `beeper-api` deployment in the `workshop-support` namespace. Set the port to 443 and the target port to 8080.

3.5. Use the `oc create route` command to create a passthrough route named `beeper-api-route` that exposes the `beeper-api` service in the `workshop-support` namespace. Use `beeper-api.apps.lab.example.com` as the hostname.

4. Verify the `beeper-api` application.

4.1. Wait until the deployments and pods for the `beeper-db` and `beeper-api` applications are ready. The deployments and pods are created in the `workshop-support` namespace.

4.2. Connect to the database container and verify that the `beeper` database exists.

```
[student@workstation ~]$ oc exec deployment/beeper-db -n workshop-suppor
t \
-- psql -e --list | grep -E 'Name|beeper'
   Name    |   Owner    | Encoding | Locale Provider ...
 beeper    | beeper-api | UTF8     | libc            ...
```

4.3. List the contents of the `beep` table in the `beeper` database.

```
[student@workstation ~]$ oc exec deployment/beeper-db -n workshop-suppor
t \
-- psql -d beeper -c "SELECT * FROM beep;"
 id | content | creation_date | username | votes
----+---------+---------------+----------+-------
(0 rows)  (1)
```

(1) The database table is empty.

4.4. Create a debug pod in the `workshop-support` namespace. Use the `nc -vz` command to verify that the `beeper-db` service is reachable via TCP on port 5432.

4.5. Use the `nc -vz` command from the `workstation` machine to verify that the `beeper-api` route is reachable via TCP on port 443.

4.6. **Optional**: Verify the SSL certificate by using the `openssl s_client` command. Use `beeper-api.apps.lab.example.com` as the hostname and the custom CA certificate from the `~/DO0019L/labs/compreview-apps/beeper-api/certs/ca.pem` file.

> **NOTE**
>
> The output of the `openssl s_client` command is large and is trimmed in sections for better readability.

```
[student@workstation ~]$ openssl s_client -CAfile \
~/DO0019L/labs/compreview-apps/beeper-api/certs/ca.pem \
-connect beeper-api.apps.lab.example.com:443 < /dev/null  (1)
depth=1 C = US, ST = North Carolina, L = Raleigh, O = Red Hat,
CN = lab.example.com
verify return:1
depth=0 C = US, ST = North Carolina, L = Raleigh, O = Red Hat,
CN = beeper-api.apps.lab.example.com
verify return:1
---
Certificate chain
 0 s:C = US, ST = North Carolina, L = Raleigh, O = Red Hat,
     CN = beeper-api.apps.lab.example.com
   i:C = US, ST = North Carolina, L = Raleigh, O = Red Hat,
     CN = lab.example.com
   a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
   v:NotBefore: Sep 28 00:11:22 2025 GMT; NotAfter: Sep 28 00:11:22 2030
GMT
---
```

(1) Redirect STDIN from `/dev/null` to avoid keyboard input.

```
Server certificate
-----BEGIN CERTIFICATE-----
...output omitted...
-----END CERTIFICATE-----
subject=C = US, ST = North Carolina, L = Raleigh, O = Red Hat,
        CN = beeper-api.apps.lab.example.com  (1)
issuer=C = US, ST = North Carolina, L = Raleigh, O = Red Hat,
       CN = lab.example.com
---
No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
---
```

(1) The ingress returns the correct SSL certificate for the route.

```
SSL handshake has read 1684 bytes and written 416 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)  (1)
---
DONE
...output omitted...
```

(1) The server SSL certificate is verified correctly by using the custom CA certificate.

4.7. Verify that the web application endpoint is working correctly by using the `curl` command. Use the `--cacert` option to accept the `~/DO0019L/labs/compreview-apps/beeper-api/certs/ca.pem` custom CA certificate.

```
[student@workstation ~]$ curl -vfS -w '\n' \
--cacert ~/DO0019L/labs/compreview-apps/beeper-api/certs/ca.pem \
https://beeper-api.apps.lab.example.com/api/beeps
*   Trying 192.168.50.254:443...
* Connected to beeper-api.apps.lab.example.com (192.168.50.254) port 443
(#0)
* ALPN, offering h2
* ALPN, offering http/1.1
*  CAfile: beeper-api/certs/ca.pem  (1)
...output omitted...
* Server certificate:
*  subject: C=US; ST=North Carolina; L=Raleigh; O=Red Hat;
            CN=beeper-api.apps.lab.example.com
*  start date: Sep 28 00:11:22 2025 GMT
*  expire date: Sep 28 00:11:22 2030 GMT
*  subjectAltName: host "beeper-api.apps.lab.example.com" matched cert's
                   "*.apps.lab.example.com"
*  issuer: C=US; ST=North Carolina; L=Raleigh; O=Red Hat; CN=lab.exampl
e.com
*  SSL certificate verify ok.  (2)
...output omitted...
* Connection #0 to host beeper-api.apps.lab.example.com left intact
[]  (3)
```

(1) The custom CA is used to verify the server SSL certificate.

(2) The server SSL certificate is verified correctly by using the custom CA certificate.

3 The command returns an empty JSON array (`[]`) because the database table is empty without messages.

4.8. Create a message with a POST request to the `beeper-api` application by using the `curl` command. This request creates an entry in the database.

```
[student@workstation ~]$ curl -vfS -w '\n' \
--cacert ~/DO0019L/labs/compreview-apps/beeper-api/certs/ca.pem \
-H 'Content-Type: application/json' \
-d '{ "username": "my-user", "content": "my content" }' \
https://beeper-api.apps.lab.example.com/api/beep
*   Trying 192.168.50.254:443...
* Connected to beeper-api.apps.lab.example.com (192.168.50.254) port 443
(#0)
...output omitted...
> POST /api/beep HTTP/1.1   1
> Host: beeper-api.apps.lab.example.com
...output omitted...
< HTTP/1.1 200   2
...output omitted...
* Connection #0 to host beeper-api.apps.lab.example.com left intact
```

1 The HTTP POST request is sent with the specified data.

2 The server responds a successful status message.

4.9. Verify that the message is stored in the database.

```
[student@workstation ~]$ oc exec deployment/beeper-db -n workshop-suppor
t \
-- psql -d beeper -c "SELECT * FROM beep;"
 id |  content   |      creation_date      | username | votes
----+------------+-------------------------+----------+-------
  1 | my content | 2026-08-11 06:33:52.553 | my-user  |     0
(1 row)
```

4.10. Confirm that the API retrieves the message from the database by using the `curl` command.

```
[student@workstation ~]$ curl -s \
--cacert ~/DO0019L/labs/compreview-apps/beeper-api/certs/ca.pem \
https://beeper-api.apps.lab.example.com/api/beeps | jq .
[
  {
    "id": 1,
    "username": "my-user",
    "content": "my content",
    "votes": 0
  }
]
```

5. Apply the network policy to the database application.

   5.1. Edit the ~/DO0019L/labs/compreview-apps/beeper-api/networkpolicy-beeper-db.yaml file to prepare the database network policy. This network policy restricts the access to the beeper-db service to pods with the app=beeper-api label and that are in a namespace with the category=support label. You applied the category=support label to the workshop-support namespace in a previous step.

   ```
   [student@workstation ~]$ oc apply -n workshop-support \
   -f ~/DO0019L/labs/compreview-apps/beeper-api/networkpolicy-beeper-db.yaml
   ...output omitted...
   ```

   5.2. Verify that you cannot connect to the database service on port 5432 because the debug pod does not have the required label. Use a debug pod to run the nc -vz command.

6. Apply the network policy to the API application.

   6.1. Edit the ~/DO0019L/labs/compreview-apps/beeper-api/networkpolicy-beeper-api.yaml file to prepare the network policy for the beeper-api application. This network policy restricts the access to the beeper-api service to pods from namespaces with the policy-group.network.openshift.io/ingress="" label. The network policy accepts ingress connections from router pods.

   6.2. Verify that you cannot connect to the API service on port 443 from the default namespace because the namespace does not have the required label. Use a debug pod to run the nc -vz command.

   6.3. Confirm that the API application is still accessible by using the route and the custom CA certificate.

   ```
   [student@workstation ~]$ curl -s -w '\n' \
   --cacert ~/DO0019L/labs/compreview-apps/beeper-api/certs/ca.pem \
   https://beeper-api.apps.lab.example.com/api/beeps
   [{"id":1,"username":"my-user","content":"my content","votes":0}]
   ```

Hide Solution

Evaluation

As the student user on the workstation machine, use the lab command to grade your work. Correct any reported failures and rerun the command until successful.

> **NOTE**
>
> The grading scripts provide remediation hints to help you identify what content you must revisit to successfully complete the tasks.

```
[student@workstation ~]$ lab grade compreview-apps
```

Evaluate the allocated time for this activity against the time that you took to complete it. If you used more time than the allocated time, then you must revisit the course content again, or practice more.

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish compreview-apps
```

Revision: do280-4.22-ed1-20260827-78c07d2



Privacy Policy (https://www.redhat.com/en/about/privacy-policy?extIdCarryOver=true&sc_cid=701f2000001D8QoAAK)

Red Hat Training Policies (https://www.redhat.com/en/about/red-hat-training-policies)

Terms of Use (https://www.redhat.com/en/about/terms-use)

All policies and guidelines (https://www.redhat.com/en/about/all-policies-guidelines)

Release Notes (https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Cookie Preferences and Opt-Out Rights